

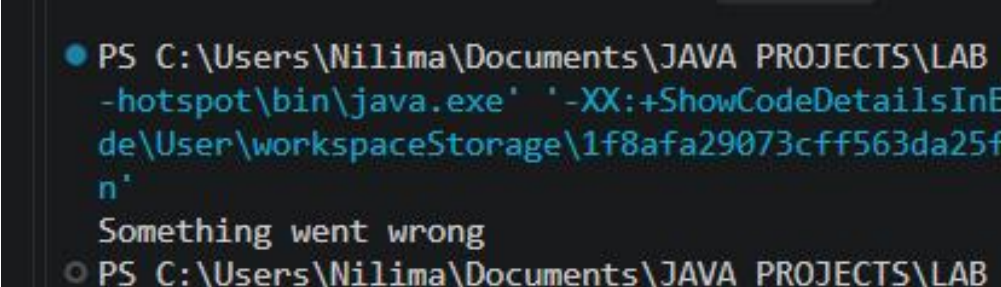


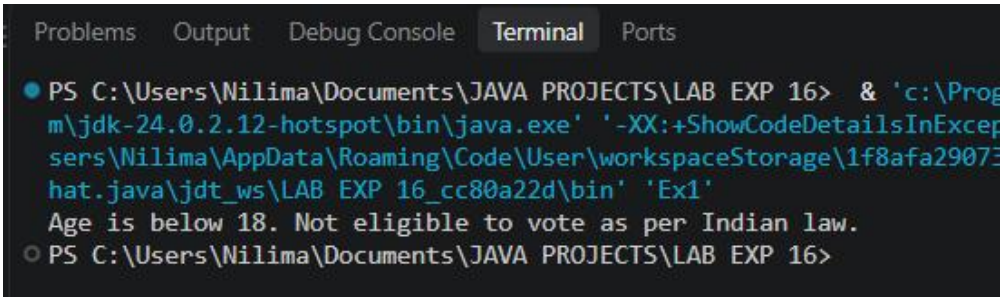
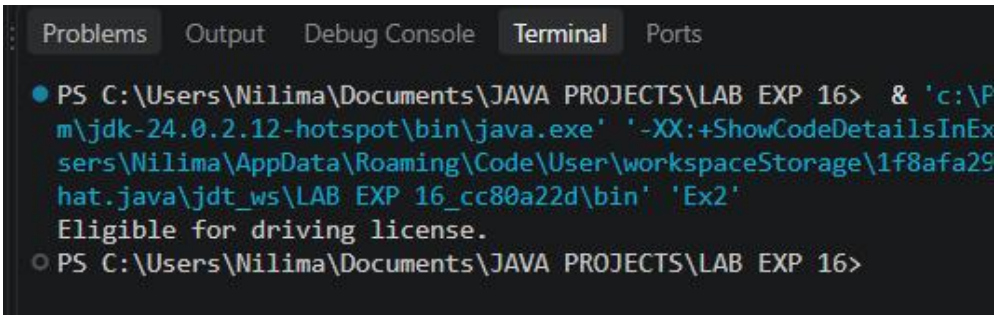
Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 16	
Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Write a Java program to create and handle a user-defined exception.
Problem Statement & Learning Objective	Problem Statement Write a Java program to create and handle a user-defined exception. Learning Objectives To create and implement custom exceptions for application-specific error handling.
Theory	User-defined exceptions are created by extending the Exception class. They allow developers to define meaningful exceptions for specific business logic. Custom exceptions improve program readability and simplify debugging by providing descriptive error messages. They are widely used in enterprise software and validation systems.
Program Code	<pre>class MyException extends Exception { MyException(String message) { super(message); } } public class Main { public static void main(String[] args) { try { throw new MyException("Something went wrong"); } } }</pre>

	<pre> } catch (MyException e) { System.out.println(e.getMessage()); } } } </pre>
Output Screenshots	 <pre> PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB -hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInE de\User\workspaceStorage\1f8afa29073cff563da25f n' Something went wrong PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB </pre>
Exercise Qns and Code	1. Create a Voting System that throws a custom exception if age is below 18. <pre> class InvalidAgeException extends Exception { InvalidAgeException(String message) { super(message); } } </pre> <pre> public class Exp1 { public static void main(String[] args) { int age = 16; </pre> <pre> try { if (age < 18) { throw new InvalidAgeException("Age is below 18. Not eligible to vote as per Indian law."); } </pre> <pre> System.out.println("Eligible to vote."); } catch (InvalidAgeException e) { System.out.println(e.getMessage()); } } } </pre> 2. Create a Driving License System that throws a custom exception if the user's age is below 18. If the age is valid, display that the user is eligible for a driving license <pre> class InvalidAgeException extends Exception { InvalidAgeException(String message) { super(message); } } </pre>

	<pre> public class Exp2 { public static void main(String[] args) { int age = 20; try { if (age < 18) { throw new InvalidAgeException("Age is below 18. Not eligible for driving license."); } System.out.println("Eligible for driving license."); } catch (InvalidAgeException e) { System.out.println(e.getMessage()); } } } </pre>
Output Screenshots	Age = 16  Age = 20 
Conclusion	This experiment introduces application-specific exception types built by extending standard exception classes. It highlights how custom exceptions provide clearer domain error semantics, improved debugging traceability, and cleaner validation logic for business rules, like eligibility and transaction checks